Version: 1.0



Selection

C-Pointers

Résumé

Entraînez-vous aux opérations fondamentales sur les pointeurs en C, y compris l'affectation de valeurs, l'échange, les opérations arithmétiques et la manipulation de tableaux.

#C

#Pointers

#Memory-Management



Avertissement relatif à la propriété intellectuelle

L'ensemble du contenu présenté dans ce module de formation — y compris, sans s'y limiter, les textes, images, graphiques et autres éléments — est protégé par les droits de propriété intellectuelle détenus par l'Association 42.

Conditions d'utilisation

- **Usage personnel :** L'utilisation du contenu de ce module est strictement réservée à un usage personnel. Toute reproduction, diffusion, modification, utilisation publique ou commerciale est interdite sans l'autorisation écrite préalable de l'Association 42.
- **Respect de l'intégrité :** Il est interdit de modifier, altérer ou adapter le contenu de manière à en dénaturer le sens ou à porter atteinte à son intégrité.

Protection des droits

Toute violation des présentes conditions constitue une atteinte aux droits de propriété intellectuelle et peut faire l'objet de poursuites. L'Association 42 se réserve le droit d'engager toute action nécessaire à la protection de ses droits, notamment par voie judiciaire et en réclamant des dommages et intérêts le cas échéant.

Pour toute question relative à l'utilisation du contenu ou pour obtenir une autorisation, veuillez contacter : `legal@42.fr`

Table des matières

1 Instructions 1

2 Préambule 5

3 Tutoriel 6

4 Exercice 0 : ft_ft 9

5 Exercice 1 : ft_ultimate_ft 10

6 Exercice 2 : ft_swap 11

7 Exercice 3 : ft_div_mod 12

8 Exercice 4 : ft_ultimate_div_mod 13

9 Exercice 5 : ft_rev_int_tab 14

10 Exercice 6 : ft_sort_int_tab 15

11 Soumission et évaluation par les pairs 16

Chapitre 1

Instructions

- Seule cette page servira de référence : ne faites pas confiance aux rumeurs.
- Attention ! Ce document pourrait potentiellement changer jusqu'à la soumission.
- Assurez-vous d'avoir les permissions appropriées sur vos fichiers et répertoires.
- Vous devez suivre les procédures de soumission pour tous vos exercices.
- Vos exercices seront vérifiés et notés par vos camarades de classe.
- De plus, vos exercices seront vérifiés et notés par un programme appelé Moulinette.
- Moulinette est très méticuleuse et stricte dans son évaluation de votre travail. Elle est entièrement automatisée, et il n'y a aucun moyen de négocier avec elle. Donc, pour éviter les mauvaises surprises, soyez aussi minutieux que possible.
- Moulinette n'est pas très ouverte d'esprit. Elle n'essayera pas de comprendre votre code s'il ne respecte pas la Norme. Moulinette s'appuie sur un programme appelé `norminette` pour vérifier si vos fichiers respectent la norme. En bref : il serait stupide de soumettre un travail qui ne passe pas la vérification de `norminette`.
- Ces exercices sont soigneusement disposés par ordre de difficulté - du plus facile au plus difficile. Nous `ne considérerons pas` un exercice plus difficile réussi si un exercice plus facile n'est pas parfaitement fonctionnel.
- Utiliser une fonction interdite est considéré comme de la tricherie. Les tricheurs obtiennent -42, et cette note est non négociable.
- Vous n'aurez à soumettre une fonction main() que si nous demandons un programme.
- Moulinette compile avec ces flags : -Wall -Wextra -Werror, et utilise `cc`.
- Si votre programme ne compile pas, vous obtiendrez `0`.
- Vous ne pouvez pas laisser aucun fichier supplémentaire dans votre répertoire autre que ceux spécifiés dans le sujet.
- Vous avez une question ? Demandez à votre pair à droite. Sinon, essayez votre pair à gauche.
- Votre guide de référence s'appelle `Google / man / Internet / ...`.
- Consultez le Slack Piscine.
- Examinez attentivement les exemples. Ils pourraient très bien appeler à des détails qui ne sont pas explicitement mentionnés dans le sujet...

© 2026 Association 42. - Tous droits réservés

- Par Odin, par Thor ! Utilisez votre cerveau ! ! !



N'oubliez pas d'ajouter l'*en-tête standard 42* dans chacun de vos fichiers .c/.h. Norminette vérifie son existence de toute façon !



Norminette doit être lancée avec le flag *-R CheckForbiddenSourceHeader*. Moulinette l'utilisera aussi.

© 2026 Association 42. - Tous droits réservés

Contexte

La Piscine C est intense. C'est votre premier grand défi à 42 — une plongée profonde dans la résolution de problèmes, l'autonomie et la communauté.

Durant cette phase, votre objectif principal est de construire vos fondations — à travers la lutte, la répétition, et surtout l'échange d'**apprentissage par les pairs**.

À l'ère de l'IA, les raccourcis sont faciles à trouver. Cependant, il est important de considérer si votre utilisation de l'IA vous aide vraiment à grandir — ou si elle se contente de vous empêcher de développer de vraies compétences.

La Piscine est aussi une expérience humaine — et pour l'instant, rien ne peut remplacer cela. Pas même l'IA.

Pour un aperçu plus complet de notre position sur l'IA — en tant qu'outil d'apprentissage, dans le cadre du programme ICT, et en tant qu'attente croissante sur le marché du travail — veuillez vous référer à la FAQ dédiée disponible sur l'intranet.

Message principal

- Construire des fondations solides sans raccourcis.
- Développer réellement des compétences techniques et personnelles.
- Expérimenter un véritable apprentissage par les pairs, commencer à apprendre à apprendre et à résoudre de nouveaux problèmes.
- Le parcours d'apprentissage est plus important que le résultat.
- Apprendre les risques associés à l'IA, et développer des pratiques de contrôle efficaces et des contre-mesures pour éviter les pièges courants.

Règles pour les apprenants :

- Vous devez appliquer le raisonnement à vos tâches assignées, surtout avant de vous tourner vers l'IA.
- Vous ne devez pas demander de réponses directes à l'IA.
- Vous devez apprendre l'approche globale de 42 sur l'IA.

© 2026 Association 42. - Tous droits réservés

Résultats de la phase :

Dans cette phase de fondation, vous obtiendrez les résultats suivants :

- Acquérir des bases techniques et de codage appropriées.
- Savoir pourquoi et comment l'IA peut être dangereuse pendant cette phase.

Commentaires et exemple :

- Oui, nous savons que l'IA existe — et oui, elle peut résoudre vos activités. Mais vous êtes ici pour apprendre, pas pour prouver que l'IA a appris. Ne gaspillez pas votre temps (ou le nôtre) juste pour démontrer que l'IA peut résoudre le problème donné.
- Apprendre à 42 ne consiste pas à connaître la réponse — il s'agit de développer la capacité à en trouver une. L'IA vous donne la réponse directement, mais cela vous empêche de construire votre propre raisonnement. Et le raisonnement prend du temps, des efforts, et implique des échecs. Le chemin vers le succès n'est pas censé être facile.
- Gardez à l'esprit que lors des examens, l'IA n'est pas disponible — pas d'internet, pas de smartphones, etc. Vous réaliserez rapidement si vous avez trop compté sur l'IA dans votre processus d'apprentissage.
- L'apprentissage par les pairs vous expose à différentes idées et approches, améliorant vos compétences interpersonnelles et votre capacité à penser de manière divergente. C'est bien plus précieux que de simplement discuter avec un bot. Alors ne soyez pas timide — parlez, posez des questions, et apprenez ensemble !
- Oui, l'IA fera partie du programme — à la fois comme outil d'apprentissage et comme sujet en soi. Vous aurez même l'occasion de construire votre propre logiciel d'IA. Afin d'en apprendre davantage sur notre approche crescendo, vous passerez par la documentation disponible sur l'intranet.

✓ Bonne pratique :

> Je suis bloqué sur un nouveau concept. Je demande à quelqu'un à proximité comment il l'a abordé. Nous parlons pendant 10 minutes — et soudain, ça fait clic. Je comprends.

✗ Mauvaise pratique :

> J'utilise secrètement l'IA, je copie du code qui semble correct. Pendant l'évaluation par les pairs, je ne peux rien expliquer. J'échoue. Pendant l'examen — sans IA — je suis à nouveau bloqué. J'échoue.

© 2026 Association 42. - Tous droits réservés

Chapitre 2

Préambule

La première mention connue du jeu de pierre-feuille-ciseaux apparaît dans le livre *Wuzazu*, écrit par l'auteur chinois de la dynastie Ming, Xie Zhaozhi, qui indiquait que le jeu remontait à la dynastie Han (206 av. J.-C. – 220 ap. J.-C.). Le jeu y était appelé *shoushiling*. Le livre *Note de Liuyanzhai* de Li Rihua mentionne aussi ce jeu sous les noms *shoushiling*, *huozhitou* ou *huoquan*.

Dans l'histoire japonaise, on trouve de fréquentes références à *sansukumi-ken*, signifiant des jeux de poings (*ken*) avec une impasse à trois (*san* signifiant trois, *sukumi* signifiant impasse). Cela signifie que A bat B, B bat C, et C bat A. Ces jeux sont originaires de Chine, ont été importés au Japon puis y sont devenus populaires.

Au début du XXe siècle, pierre-feuille-ciseaux s'était déjà répandu au-delà de l'Asie, notamment grâce aux échanges accrus entre le Japon et l'Occident. Son nom en anglais est une traduction des gestes japonais pour pierre, feuille et ciseaux. Dans d'autres pays d'Asie, le geste de la main ouverte représente le "tissu" plutôt que la "feuille". La forme des ciseaux est également héritée du style japonais.

En 1927, le magazine pour enfants *La Vie au patronage*, en France, le décrit en détail en l'appelant "jeu japonais". Son nom français, "Chi-fou-mi", vient des anciens mots japonais pour "un, deux, trois" ("hi, fu, mi").

Un article du New York Times en 1932 sur l'heure de pointe à Tokyo explique les règles du jeu pour les lecteurs américains, laissant entendre qu'il était encore peu connu aux États-Unis. L'encyclopédie *Compton's Pictured Encyclopedia* de 1933 le décrit comme une méthode courante de résolution de conflits entre enfants dans son article sur le Japon ; le nom y est donné comme "John Kem Po", avec une suggestion appuyée : "C'est une si bonne manière de régler un désaccord que les garçons et filles américains pourraient vouloir l'essayer eux aussi."

© 2026 Association 42. - Tous droits réservés

Chapitre 3

Tutoriel

- Il est maintenant temps de parler de la partie magique de la programmation en C : **LES POINTEURS** ! Un **pointeur** est un type spécial en `C` qui contient une adresse. Chaque octet en RAM a sa propre adresse individuelle, et un pointeur stocke l'adresse d'un autre octet en RAM.
- Un `pointeur` n'est pas autonome - vous avez toujours un pointeur vers un autre type `C`. Exemple : vous pouvez avoir un pointeur sur un caractère, ou un pointeur sur un entier. Un pointeur sur un entier stocke l'adresse d'un entier existant ailleurs en mémoire.
- Vous devriez déclarer un pointeur comme suit : `<type> *<nom_var>`. Par exemple : `int *pointeur_sur_entier;`



Un pointeur ne contient pas une valeur comme les autres types. Il contient une adresse où une valeur est stockée. Si vous ne comprenez pas pleinement, vous devez vérifier votre compréhension avec l'un de vos voisins.

- Créez un répertoire de travail et écrivez un fichier `C` simple. Déclarez un pointeur sur un entier et compilez-le pour vérifier la syntaxe :

main.c

```
int main()
{
    int *pointeur_sur_entier;
    return(0);
}
```

- Vous pouvez assigner une adresse au pointeur en utilisant l'opérateur `&` pour obtenir l'adresse d'une variable existante :

© 2026 Association 42. - Tous droits réservés

Exemple

```
int a;
int *p;
p = &a;
```

- Ajoutez une variable `int`, un pointeur `int`, et assignez l'adresse de l'`int` au pointeur. Compilez et testez.
- Vous pouvez utiliser le pointeur pour lire et mettre à jour l'autre variable. Cela s'appelle le "déréférencement" en utilisant l'opérateur `*`. Par exemple, `*p` peut être utilisé comme une variable `int` régulière.
- Assignez une valeur à votre variable `int` et imprimez-la en utilisant votre fonction `ft_putnbr`.
- Maintenant, imprimez la même valeur mais en utilisant le pointeur au lieu du nom de la variable : `ft_putnbr(*p);`. Vérifiez que les deux impressions montrent la même valeur.
- Mettez à jour l'`int` en utilisant le nom de la variable, puis imprimez à nouveau en utilisant à la fois la variable et le pointeur. Qu'observez-vous ?
- Mettez à jour l'`int` en utilisant le pointeur : `*p = 100;` Puis imprimez à nouveau en utilisant les deux méthodes. Que se passe-t-il ?



Rendons cela simple pour `int *p;` :
`*p` est une variable de type `int` dont l'adresse est stockée dans `p`.

- Maintenant, explorons les `chaînes de caractères`. En `C`, une chaîne de caractères est stockée comme une série d'octets en RAM, chacun contenant la valeur `ASCII` de chaque caractère. La chaîne se termine par un octet contenant 0.
- Vous pouvez créer une chaîne de caractères en utilisant des guillemets doubles : `"Bonjour"`. Cela trouve un espace mémoire et stocke les codes ASCII. L'expression évalue à l'adresse du premier caractère.
- Déclarez un pointeur `char` et assignez-lui une `chaîne de caractères` :

Exemple

```
char *s;

s = "Bonjour";
```

- Utilisez `*s` pour accéder au premier `char` et imprimez-le avec `ft_putchar(*s);`.
- Voici l'« arithmétique des pointeurs » ! Si `s` pointe vers le premier `char`, alors `s+1` pointe vers le deuxième `char`. Essayez ce qui suit :

© 2026 Association 42. - Tous droits réservés

Exemple

```
char *s;

s = "Bonjour";
s = s + 1;
ft_putchar(*s);
```

- Maintenant, essayez :

Exemple

```
char *s;

s = "Bonjour";
ft_putchar(*(s+1));
```

Vérifiez avec l'un de vos pairs si vous voyez une différence.

- L'arithmétique des pointeurs dépend du type pointé. `char_ptr+1` pointe vers le `char` suivant, `int_ptr+1` pointe vers l'`int` suivant (4 octets plus loin).
- Expérimentez en parcourant une chaîne de caractères caractère par caractère en utilisant l'arithmétique des pointeurs. Imprimez chaque caractère individuellement.
- Essayez ce mystère : écrivez une fonction qui reçoit un pointeur `int` en paramètre et modifie la valeur vers laquelle il pointe. Appelez cette fonction depuis le main et vérifiez que la variable originale a été modifiée.

© 2026 Association 42. - Tous droits réservés

Chapitre 4

Exercice 0 : ft_ft

	Exercice: 0	
ft_ft		
Répertoire: *ex0/*		
Fichiers à Rendre: `ft_ft.c`		
Autorisé: `None`		

- Créez une fonction qui prend un pointeur sur un `int` en paramètre, et affecte la valeur "42" à ce `int`.

Prototype :

```
void ft_ft(int *nbr);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 5

Exercice 1 : ft_ultimate_ft

	Exercice: 1	
ft_ultimate_ft		
Répertoire: *ex1/*		
Fichiers à Rendre: `ft_ultimate_ft.c`		
Autorisé: `None`		

- Créez une fonction qui prend un pointeur de pointeur de pointeur de pointeur de pointeur de pointeur de pointeur de pointeur de pointeur sur un `int` en paramètre, et affecte la valeur "42" à ce `int`.

Prototype :

```
void ft_ultimate_ft(int *********nbr);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 6

Exercice 2 : ft_swap

	Exercice: 2	
ft_swap		
Répertoire: *ex2/*		
Fichiers à Rendre: `ft_swap.c`		
Autorisé: `None`		

- Créez une fonction qui échange la valeur de deux entiers dont les adresses sont passées en paramètre.

Prototype :

```
void ft_swap(int *a, int *b);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 7

Exercice 3 : ft_div_mod

	Exercice: 3	
ft_div_mod		
Répertoire: *ex3/*		
Fichiers à Rendre: `ft_div_mod.c`		
Autorisé: `None`		

- Créez une fonction `ft_div_mod`.

Prototype :

```
void ft_div_mod(int a, int b, int *div, int *mod);
```

- Cette fonction divise le paramètre `a` par `b` et stocke le résultat dans l'entier pointé par `div`. Elle stocke également le reste de la division de `a` par `b` dans l'entier pointé par `mod`.

© 2026 Association 42. - Tous droits réservés

Chapitre 8

Exercice 4 : ft_ultimate_div_mod

	Exercice: 4	
ft_ultimate_div_mod		
Répertoire: *ex*4/		
Fichiers à Rendre: `ft_ultimate_div_mod.c`		
Autorisé: `None`		

- Créez une fonction `ft_ultimate_div_mod`.

Prototype :

```
void ft_ultimate_div_mod(int *a, int *b);
```

- Cette fonction divise le contenu pointé par `a` par celui pointé par `b`.
  - Le résultat est stocké dans l'entier pointé par `a`.
  - Le reste de la division est stocké dans l'entier pointé par `b`.

© 2026 Association 42. - Tous droits réservés

Chapitre 9

Exercice 5 : ft_rev_int_tab

	Exercice: 5	
	ft_rev_int_tab	
	Répertoire: *ex*5/	
	Fichiers à Rendre: `ft_rev_int_tab.c`	
	Autorisé: `None`	

- Créez une fonction qui inverse un tableau d'entiers donné (le premier devient le dernier, etc).

Prototype :

```
void ft_rev_int_tab(int *tab, int size);
```

- Les arguments sont un pointeur sur un `int` et le nombre d'éléments dans le tableau.

© 2026 Association 42. - Tous droits réservés

Chapitre 10

Exercice 6 : ft_sort_int_tab

	Exercice: 6	
ft_sort_int_tab		
Répertoire: *ex6/*		
Fichiers à Rendre: `ft_sort_int_tab.c`		
Autorisé: `None`		

- Créez une fonction qui trie un tableau d'entiers par ordre croissant.

Prototype :

```
void ft_sort_int_tab(int *tab, int size);
```

- Les arguments sont un pointeur sur un `int` et le nombre d'éléments dans le tableau.

© 2026 Association 42. - Tous droits réservés

Chapitre 11

Soumission et évaluation par les pairs

Déposez votre devoir dans votre dépôt `Git` comme d'habitude. Seul le contenu présent dans votre dépôt sera évalué lors de la soutenance. N'hésitez pas à vérifier les noms de vos fichiers pour vous assurer qu'ils sont corrects.



Vous devez rendre uniquement les fichiers demandés par l'énoncé de ce projet.

© 2026 Association 42. - Tous droits réservés